Безопасность и управление доступом

Table of contents

Безопасность и управление доступом 1
Аутентификация vs Авторизация 1
RBAC (Role-Based Access Control) 2
ABAC (Attribute-Based Access Control) 3
JWT и OAuth 2.0 5
Single Sign-On (SSO) 7

Безопасность и управление доступом

После решения проблемы распределения нагрузки переходим к **безопасности** — критическому аспекту любой продакшен-системы.

Когда ваше приложение выходит в интернет, возникают вопросы: **кто может получить доступ к данным? Как проверить, что пользователь — это действительно он? Как контролировать, что он может делать?** Без механизмов безопасности любой пользователь может удалить чужие данные, получить доступ к приватной информации или вообще подменить свою личность.

Эта лекция охватывает три уровня защиты: 1. **Аутентификация и авторизация** (RBAC, ABAC) — проверка личности и прав доступа 2. **Токены и федеративная аутентификация** (JWT, OAuth 2.0) — как передавать права между системами 3. **Single Sign-On (SSO)** — как пользователи логинятся один раз для доступа ко многим сервисам (SAML, OpenID Connect)

В больших организациях (Google, Microsoft) сотрудники не вводят пароль для каждого внутреннего сервиса — они логинятся через корпоративный SSO. Социальные сети используют OAuth, чтобы сторонние приложения могли получить доступ к вашим данным без передачи пароля. Эта лекция объясняет, как всё это работает.

Аутентификация vs Авторизация

Аутентификация (Authentication): Кто вы? - Проверка личности пользователя - Методы: пароль, OAuth, биометрия, 2FA

Авторизация (Authorization): Что вам разрешено делать? - Проверка прав доступа - Методы: RBAC, ABAC, ACL

Пример:

```
1. Аутентификация: пользователь alice@company.com вводит пароль → система
проверяет → выдаёт токен
```

```
2. Авторизация: alice запрашивает DELETE /api/users/123
   → система проверяет: у alice есть роль admin?
   → если да → разрешить, иначе → 403 Forbidden
```

RBAC (Role-Based Access Control)

Основная идея: права доступа назначаются **ролям**, пользователи получают роли.

Структура:

```
Пользователи → Роли → Права (Permissions)

Пример:
  Alice → Role: Editor → Permissions: [read_posts, write_posts, delete_own_posts]
  Bob   → Role: Admin  → Permissions: [read_posts, write_posts, delete_any_posts,
manage_users]
  Carol → Role: Viewer → Permissions: [read_posts]
```

Пример: система управления контентом

Роли и права:

```go
type Permission string

const (
    ReadPosts      Permission = "posts:read"
    WritePosts     Permission = "posts:write"
    DeleteOwnPosts Permission = "posts:delete:own"
    DeleteAnyPosts Permission = "posts:delete:any"
    ManageUsers    Permission = "users:manage"
)

type Role struct {
    Name        string
    Permissions []Permission
}

var roles = map[string]Role{
    "viewer": {
        Name:        "Viewer",
        Permissions: []Permission{ReadPosts},
    },
    "editor": {
        Name:        "Editor",
        Permissions: []Permission{ReadPosts, WritePosts, DeleteOwnPosts},
```

```
    },
    "admin": {
        Name:        "Admin",
             Permissions:  []Permission{ReadPosts,  WritePosts,  DeleteOwnPosts,
DeleteAnyPosts, ManageUsers},
    },
}
```

Проверка доступа:

```go
func (u *User) HasPermission(perm Permission) bool {
    role := roles[u.Role]
    for _, p := range role.Permissions {
        if p == perm {
            return true
        }
    }
    return false
}

// Пример использования
if !user.HasPermission(DeleteAnyPosts) {
    return errors.New("403 Forbidden")
}
```

Преимущества: - Простота управления (меняем права роли → изменения для всех пользователей) - Легко понять структуру прав

Недостатки: - Не поддерживает динамические правила (например, “редактировать только свои посты”) - Взрыв ролей при сложных требованиях (нужна отдельная роль для каждой комбинации прав)

Где используется: большинство веб-приложений, CMS, корпоративные системы

ABAC (Attribute-Based Access Control)

Основная идея: доступ определяется **атрибутами** субъекта, ресурса и контекста.

Структура:

```
Субъект (User) + Ресурс (Resource) + Действие (Action) + Контекст (Context)
→ Политика (Policy) → Решение (Allow/Deny)
```

Пример: политики доступа

Атрибуты:

```json
{
  "subject": {
    "id": "alice",
    "department": "engineering",
    "role": "senior",
    "clearance_level": 3
  },
  "resource": {
    "type": "document",
    "id": "doc-123",
    "classification": "confidential",
    "owner": "alice",
    "department": "engineering"
  },
  "action": "edit",
  "context": {
    "time": "2025-01-02T14:30:00Z",
    "ip": "192.168.1.10",
    "mfa_verified": true
  }
}
```

Политики (примеры):

```
Политика 1: Пользователь может редактировать свои документы
  IF subject.id == resource.owner
  AND action == "edit"
  THEN Allow

Политика 2: Старшие сотрудники могут читать конфиденциальные документы своего
департамента
  IF subject.role == "senior"
  AND subject.department == resource.department
  AND resource.classification == "confidential"
  AND action == "read"
  THEN Allow

Политика 3: Доступ к секретным документам только в рабочее время с MFA
  IF resource.classification == "secret"
  AND subject.clearance_level >= 3
  AND context.time BETWEEN 09:00 AND 18:00
  AND context.mfa_verified == true
```

```
  THEN Allow
  ELSE Deny
```

Преимущества: - Гибкость: можно выразить сложные правила - Динамические проверки (время, IP, MFA) - Меньше “взрыва” ролей

Недостатки: - Сложнее реализация и отладка - Производительность (нужно вычислять политики на каждый запрос)

Где используется: - Банки и финансы (сложные правила доступа к данным) - Государственные системы (классификация документов) - Healthcare (HIPAA compliance, доступ по атрибутам пациента и врача)

Сравнение RBAC и ABAC:

Характеристика	RBAC	ABAC
Основа	Роли	Атрибуты (субъект, ресурс, контекст)
Сложность	Простая	Сложная
Гибкость	Низкая	Высокая
Динамические правила	Нет	Да (время, IP, MFA и т.д.)
Производительность	Быстро (проверка в памяти)	Медленнее (вычисление политик)
Когда использовать	Простые иерархии, CMS, админки	Сложные правила, финансы, healthcare

JWT и OAuth 2.0

JSON Web Token (JWT):

Что это: токен, содержащий закодированные claims (утверждения) о пользователе, подписанный сервером.

Структура:

```
Header.Payload.Signature

Пример:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkFsaWNlIiwicm9sZSI6ImVkaXRvciIsImlhdCI6MTUxNjIzOTAyMn0.
SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Декодированный Payload:
{
```

```
  "sub": "1234567890",     // subject (user ID)
  "name": "Alice",
  "role": "editor",
  "iat": 1516239022,       // issued at (timestamp)
  "exp": 1516242622        // expiration (timestamp)
}
```

Как работает:

```
1. Клиент → POST /login (username, password)
2. Сервер проверяет credentials → генерирует JWT с claims
3. Сервер → возвращает JWT клиенту
4. Клиент сохраняет JWT (localStorage, cookie)
5. Для каждого запроса: клиент отправляет JWT в заголовке Authorization: Bearer
<token>
6. Сервер проверяет подпись JWT → извлекает claims → проверяет права
```

Преимущества: - Stateless (сервер не хранит сессии, все данные в токене) - Масштабируемость (любой сервер может проверить JWT)

Недостатки: - Нельзя отозвать токен до истечения exp (нужен blacklist) - Размер токена больше, чем session ID

OAuth 2.0 Authorization Code Flow:

Сценарий: приложение “PhotoEditor” хочет получить доступ к фотографиям пользователя в Google Drive.

Flow:

```
1. Пользователь в PhotoEditor нажимает "Подключить Google Drive"

2. PhotoEditor перенаправляет на Google:
   GET https://accounts.google.com/o/oauth2/v2/auth?
       client_id=photeditor_client_id
       &redirect_uri=https://photoeditor.com/callback
       &scope=drive.readonly
       &response_type=code

3. Пользователь видит экран согласия Google:
   "PhotoEditor хочет получить доступ к вашим файлам на Drive"
   → Пользователь нажимает "Разрешить"

4. Google перенаправляет обратно на PhotoEditor с authorization code:
   GET https://photoeditor.com/callback?code=AUTH_CODE_HERE
```

```
5. PhotoEditor (backend) обменивает code на access token:
   POST https://oauth2.googleapis.com/token
   {
     "code": "AUTH_CODE_HERE",
     "client_id": "...",
     "client_secret": "...",  // секрет, известный только backend
     "redirect_uri": "...",
     "grant_type": "authorization_code"
   }

6. Google возвращает access token:
   {
     "access_token": "ya29.a0AfH6SMC...",
     "refresh_token": "1//0gX...",
     "expires_in": 3600,
     "token_type": "Bearer"
   }

7. PhotoEditor использует access token для запросов к Google Drive API:
   GET https://www.googleapis.com/drive/v3/files
   Authorization: Bearer ya29.a0AfH6SMC...
```

Роли в OAuth 2.0: - **Resource Owner** — пользователь (владелец фотографий) - **Client** — приложение PhotoEditor - **Authorization Server** — Google (выдаёт токены) - **Resource Server** — Google Drive API (хранит фотографии)

Single Sign-On (SSO)

Что это: пользователь логинится один раз и получает доступ ко всем приложениям компании.

Пример:

```
Сотрудник входит в корпоративный портал (portal.company.com)
→ Автоматически получает доступ к:
  - Email (mail.company.com)
  - CRM (crm.company.com)
  - Jira (jira.company.com)
  - Confluence (wiki.company.com)
```

Протоколы SSO:

1. SAML 2.0 (Security Assertion Markup Language)

Enterprise-стандарт, работает через XML-сообщения.

Flow:

```
1. Пользователь → открывает CRM (Service Provider)
2. CRM видит, что пользователь не залогинен → перенаправляет на SSO (Identity
Provider)
3. SSO → показывает форму логина
4. Пользователь вводит credentials → SSO проверяет
5. SSO → создаёт SAML Assertion (XML с данными пользователя) и подписывает
6. SSO → перенаправляет обратно на CRM с SAML Assertion
7. CRM → проверяет подпись, извлекает данные пользователя → создаёт сессию
```

Где используется: корпоративные системы, интеграции B2B (Okta, Azure AD, Google Workspace)

2. OAuth 2.0 / OpenID Connect (OIDC)

Более современный, работает с JSON Web Tokens.

OpenID Connect = OAuth 2.0 + id_token (JWT с данными пользователя)

Flow:

```
1. Пользователь → открывает App
2. App → перенаправляет на Auth0 (OIDC Provider):
   GET /authorize?client_id=...&scope=openid profile email&response_type=code

3. Auth0 → показывает логин (или использует существующую сессию SSO)
4. Auth0 → перенаправляет обратно с authorization code
5. App → обменивает code на токены:
   Получает: { access_token, id_token (JWT), refresh_token }

6. App → декодирует id_token:
   {
     "sub": "user-123",
     "email": "alice@company.com",
     "name": "Alice",
     "picture": "https://...",
     "iss": "https://auth0.company.com"
   }
```

Где используется: современные SaaS-приложения, мобильные приложения (Auth0, Okta, Keycloak)

Сравнение SAML vs OAuth/OIDC:

Характеристика	SAML 2.0	OAuth 2.0 / OIDC
Формат	XML	JSON (JWT)
Сложность	Высокая	Средняя
Мобильные приложения	Плохо поддерживается	Отлично
API	RESTful wrapper нужен	Нативно RESTful
Enterprise adoption	Высокая (legacy системы)	Растёт (новые системы)
Когда использовать	B2B интеграции, enterprise SSO	SaaS, мобильные приложения, API